CSE 715: Computer Graphics

Point-Obscures-Point, Back-Face/Visible-Surface, Trapezoid-Primitive Fill

University of Chittagong — 7th Semester B.Sc. (Engg.) Examination

---

Contents

1 Point-Obscures-Point Visibility via Viewpoint (Ch10, Hidden Surfaces) 2

2 Back-Face / Visible-Surface Definitions 3

3 Trapezoid-Primitive Polygon Fill (2024 Q7b, Q7c, Q7e) 4

4 Quick Summary 5

Point-Obscures-Point Visibility via Viewpoint (Ch10, Hidden Surfaces)

Re-verified against all 4 raw scans 2026-07-07 (this topic appears 2020, 2021, 2022, 2023 — not 2024). **Method:** compute $\mathbf{P}_i - \mathbf{V}$ for each point. Two points share a line of sight from $\mathbf{V}$ iff their $(\mathbf{P} - \mathbf{V})$ vectors are **parallel with the same sign** (one is a positive scalar multiple of the other) — exactly the ray equation $\mathbf{r}(t) = \mathbf{V} + t\mathbf{d}$ from [[wiki/ray-tracing—Ch12]] evaluated at different t. Among collinear points, the **smaller** t (nearer $\mathbf{V}$) obscures the larger. If no two points are collinear, none obscures any other — all are independently visible. **2 of the 4 years actually have no occlusion at all** — don't assume every year produces a clean "obscures" chain.

2020 Q8(c) [3.5 marks]

Given points $P_1(1, 2, 0)$, $P_2(3, 6, 20)$, $P_3(2, 4, 10)$ and viewpoint $C(0, 0, -10)$, determine which points obscure the others when viewed from C.

$P_1 - C = (1, 2, 10)$, $P_2 - C = (3, 6, 30) = 3(1, 2, 10)$, $P_3 - C = (2, 4, 20) = 2(1, 2, 10)$ — **all three are collinear** with C, all positive multiples of the same direction. Ordering by distance ($t = 1, 2, 3$ respectively): P_1 is nearest, then P_3, then P_2. P_1 **obscures** P_3**, which in turn obscures** P_2 — all three lie on one ray, stacked front-to-back as P_1, P_3, P_2.

2021 Q6(a) [3 marks]

Given points $P_1(1, 3, 1)$, $P_2(3, 6, -11)$, $P_3(2, 6, -5)$ and viewpoint $C(0, 0, 7)$, determine which points obscure the others when viewed from C.

$P_1 - C = (1, 3, -6)$, $P_2 - C = (3, 6, -18)$, $P_3 - C = (2, 6, -12) = 2(1, 3, -6)$. P_3 **is collinear with** P_1 (exactly twice as far). Check P_2: $(3, 6, -18)$ vs. $3(1, 3, -6) = (3, 9, -18)$ — y-component doesn't match, so P_2 is **not** on the same line of sight as P_1/P_3. **Conclusion:** P_1 **obscures** P_3 (same ray, P_1 nearer). P_2 **is on a different line of sight** — independently visible, neither obscuring nor obscured.

2022 Q6(a) [3 marks]

Given points $A(1, 5, -2)$, $B(1, 4, 9)$, $C(-2, -2, -3)$ and viewpoint $V(0, 1, -10)$, determine which points obscure the others when viewed from V.

$A - V = (1, 4, 8)$, $B - V = (1, 3, 19)$, $C - V = (-2, -3, 7)$. Checking all three pairs for a scalar-multiple relationship: none match (e.g. A vs B: ratios $1, 1.33, 2.375$ — inconsistent; A vs C: ratios $-0.5, -1.33, 1.14$ — inconsistent; B vs C: ratios $-0.5, -1, 2.71$ — inconsistent). **Conclusion: no two of A, B, C share a line of sight from V — none obscures any other. All three are simultaneously visible.**

2023 Q6(a) [3 marks]

Given points $A(5, 1, -2)$, $B(10, 4, 9)$, $C(15, -2, -3)$ and viewpoint $V(0, 1, -10)$, determine which points obscure the others when viewed from V.

$A - V = (5, 0, 8)$, $B - V = (10, 3, 19)$, $C - V = (15, -3, 7)$. Since $A - V$ has a zero y-component, any point collinear with A must also have $P_y - V_y = 0$, i.e. $P_y = 1$ — but $B_y = 4$ and $C_y = -2$, so neither is collinear with A. Checking B vs C: $(10, 3, 19)$ vs $(15, -3, 7)$ — opposite-signed y-components rule out any positive scalar multiple. **Conclusion: no two of A, B, C share a line of sight from V — none obscures any other, same structure as 2022.**

Back-Face / Visible-Surface Definitions

2024 Q7(a) [2 marks]: Define visible surface and back-face with necessary examples. **Related — 2024 Q1(c) [3 marks]:** What is hidden surface removal in computer graphics, and what problem does it solve during image generation?

- **Visible surface:** a surface (or part of one) that is actually seen by the viewer from the current viewpoint — i.e. not blocked by any other surface along the line of sight. *Example:* the front face of a cube facing the camera.
- **Back-face:** a polygon whose outward normal points **away** from the viewer (angle between the normal $\mathbf{N}$ and the view direction $\mathbf{V}$ is > 90, i.e. $\mathbf{N} \cdot \mathbf{V} < 0$ using an outward-pointing convention). *Example:* the far side of a cube facing away from the camera. For a convex, opaque solid, every back-face is automatically hidden, so back-face detection/culling is a cheap first pass before more expensive hidden-surface algorithms (see [[wiki/z-buffer—Z-Buffer, Ch10]]).
- **Hidden-surface removal (related, Q1c):** the process of determining which surfaces (or parts of surfaces) are visible from the viewpoint and discarding/not-rendering the rest. **Problem it solves:** without it, a rendered image would show every polygon in the scene overlapping incorrectly (e.g. surfaces that should be blocked by nearer objects would still appear), producing a visually

wrong, non-realistic image — HSR ensures only what would actually be seen ends up on screen.

Trapezoid-Primitive Polygon Fill (2024 Q7b, Q7c, Q7e)

No textbook resource for this cluster — 2024 lecture-only material. The figures in the actual exam (a 6-7 vertex polygon with labeled edges E_1–E_9 for Q7c, and a separate irregular notched polygon for Q7e) are hand-drawn and only partially legible from the scan. The **method** below is what earns the marks; apply it directly to your own printed copy of each figure for the exact vertex/edge labels and trapezoid count.

Q7(b) — Steps of Trapezoid-Primitive Polygon Drawing [2 marks]

1. Sort all polygon vertices by y-coordinate; identify the distinct y-levels.
2. Draw a horizontal line through every vertex, splitting the polygon into horizontal strips between consecutive distinct y-levels.
3. Each strip is bounded on the left and right by (portions of) polygon edges — if exactly 2 edges cross a strip, that strip is one **trapezoid** (a triangle is the degenerate case where one bounding edge shrinks to a point).
4. If more than 2 edges cross a strip (can happen at a concave notch), split that strip into multiple trapezoids, one per left/right edge-pair.
5. Fill each trapezoid independently (a trapezoid's left/right boundaries are simple line segments, so each scanline within it needs only 2 boundary-intersection computations) — this is the appeal of the method: filling reduces to a sequence of simple trapezoid fills instead of one intersection-sorting pass per scanline.

Q7(c) — Vertex, Edge, and Surface Matrices [2 marks]

This is a **boundary representation**: three linked tables describing a polygon (or, in 3D, a polyhedron) structurally rather than just as a coordinate list.

- **Vertex table:** one row per vertex, storing its coordinates (e.g. $V_1(x_1, y_1), \ldots, V_n(x_n, y_n)$).
- **Edge table:** one row per edge, storing the two vertex indices it connects, plus (in 3D) pointers to the one or two surfaces bordering it (e.g. $E_1 : (V_1, V_2)$, $E_2 : (V_2, V_3)$, …, following the labels in the given figure).
- **Surface table:** one row per face, listing the ordered sequence of edges (or vertices) bounding it — for a single 2D polygon like the exam figure, one row referencing all of $E_1, \ldots, E_9$ in boundary order.

For the given figure: read off each vertex position and its connecting edges directly from the diagram (labeled E_1–E_9 around the boundary, e.g. vertex "C" at the top connects via E_2 and E_3, vertex "F" connects via E_4 and E_5, etc.) and fill the three tables in the format above — the structure is identical regardless of the specific shape.

Q7(e) — Number of Trapezoids for the Given Polygon [1 mark]

Apply the method from Q7(b): count the number of **distinct y-levels** among the polygon's vertices, draw a horizontal line through each, and count the resulting strips (adding an extra split for any strip crossed by more than 2 edges, which happens at concave notches/zigzags). **Illustrative example:** a simple convex hexagon with 4 distinct vertex y-levels produces $4-1=3$ trapezoids (no notches to force extra splits). For the exam's own (non-convex, zigzag) figure, count the vertex y-levels directly off the printed diagram and add one extra trapezoid for each notch that produces more than 2 edge-crossings in its strip.

Quick Summary

Topic	Years	Method
Point-obscures-point	2020-2023	$(P-V)$ parallel & same-sign $\Rightarrow$ same sightline; smaller t obscures larger; 2 of 4 years have *no* occlusion
Back-face/visible-surface	2024	Back-face: $\mathbf{N}\cdot\mathbf{V}<0$ (points away from viewer); visible surface: unobstructed along sightline
Trapezoid-primitive fill	2024	Horizontal strips through each vertex y-level = trapezoids; extra splits at concave notches
Vertex/edge/surface matrices	2024	3 linked tables: vertex coords, edge-to-vertex(+surface) links, surface-to-edge-loop